

DSAA3073: Theories in Data Science

Week 8: Online Learning Part 2: Regret Analysis and Algorithms

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Opening Challenge

The Adversarial Prediction Game

You're a financial analyst making daily stock predictions. You have access to n expert advisors, each providing buy/sell recommendations. Here's the catch: the market is adversarial—it can respond to your strategy to make you perform as poorly as possible.

- You cannot assume the market follows any probability distribution
- The experts themselves may be unreliable (no “perfect expert”)
- You must make decisions sequentially, one day at a time
- Your goal: perform nearly as well as the best expert in hindsight

Question: Is it even possible to compete with the best expert when the world is adversarial? If so, how close can you get?

Introduction: From Batch to Online

Online learning originated in game theory and economics, but has become central to modern machine learning applications like online advertising, recommendation systems, and algorithmic trading.

Backbone message: This week reveals a fundamental principle: **learning with uncertainty is possible through multiplicative weight updates**. We'll see how a simple exponential reweighting scheme—applied to experts, losses, or gradients—achieves optimal regret bounds without any distributional assumptions.

In the previous weeks, we studied PAC learning, where we assumed training data is drawn i.i.d. from some fixed distribution. This week, we enter a fundamentally different paradigm: **online learning** (a sequential decision-making framework where we make predictions one at a time and observe outcomes immediately), where:

- No distributional assumptions are made
- Decisions are made sequentially
- The environment can be adversarial
- Performance is measured by **regret** (the difference between our total loss and the best fixed strategy's loss in hindsight) rather than generalization error

← From Week 9: AdaBoost and Ensemble Learning

Week 9 introduced AdaBoost, which maintains weights D_t over training examples and updates them exponentially: $D_{t+1}(i) \propto D_t(i) \exp(-\alpha_t y_i h_t(x_i))$. This multiplicative update felt mysterious—why exponential weights? This week provides the answer: AdaBoost is an instance of the Multiplicative Weights Update framework, one of the most powerful ideas in online learning.

← From Week 1: PAC Learning and Sample Complexity

In Week 1, we asked: “How many samples suffice for learning?” and bounded sample complexity by $O(\frac{d}{\epsilon} \log \frac{1}{\delta})$ for VC dimension d . Online learning asks a different question: “How can we adapt sequentially to minimize regret?” The guarantees are incomparable—PAC gives probabilistic bounds on fixed data, online learning gives worst-case bounds on adaptive sequences.

Connection

From Week 9 to Week 10: Last week we studied AdaBoost, which maintains weights over training examples and updates them multiplicatively. This week, we'll see that AdaBoost is actually a special case of a more general online learning framework called Multiplicative Weights Update (MWU).

Mental Model: The Online Learning Landscape

Online learning encompasses several interconnected frameworks:

By Decision Space: $\vdash$ Discrete (Expert Advice) $\mid \vdash$ Halving Algorithm (perfect expert assumption) $\mid \vdash$ Weighted Majority (deterministic) $\mid \vdash$ Randomized Weighted Majority / MWU $\vdash$ Continuous (Convex Optimization) $\vdash$ Online Gradient Descent (OGD) $\vdash$ Follow-The-Regularized-Leader (FTRL)

Unifying Principle: All achieve $O(\sqrt{T})$ regret through multiplicative/exponential updates

Key Relationships:

- MWU with entropy regularizer $\rightarrow$ exponential weights on simplex
- FTRL with quadratic regularizer $\rightarrow$ relates to OGD
- AdaBoost = MWU applied to training examples
- SGD = OGD in stochastic setting

Performance Hierarchy:

- Perfect expert exists: $O(\log n)$ mistakes (Halving)
- Adversarial setting: $O(\sqrt{T})$ regret (optimal, unavoidable)
- Stochastic setting with gaps: $O(\log T)$ regret (with stronger assumptions)

The Online Learning Framework

Motivation: Why study online learning when we already have PAC learning? Because many real-world scenarios don't satisfy i.i.d. assumptions: adversarial opponents in games, non-stationary environments in robotics, strategic users in recommendation systems. Online learning provides guarantees even when the world actively works against us.

Definition: Online Learning Protocol

The online learning game proceeds in T rounds. In each round $t = 1, 2, \dots, T$:

1. **Learner** chooses an action/prediction
2. **Environment** reveals the outcome
3. **Learner** observes the loss and updates strategy

The environment can be adversarial—it may choose outcomes to make the learner perform poorly.

Definition: Adversarial Models

In online learning, we model the world as an **adversary** that chooses the learner's loss. The strength of the adversary determines what

Three standard adversary models (from weakest to strongest)

1. **Oblivious adversary:** The adversary commits to the entire sequence $(y_1, \dots, y_T)$ before the game begins, without any knowledge of the learner's strategy or randomization. This is the weakest adversary model.
2. **Non-anticipating (adaptive) adversary:** The adversary can choose outcome y_t adaptively based on the learner's past actions $a_1, \dots, a_{t-1}$, but cannot see the current randomization used to produce a_t . This is stronger than oblivious but weaker than fully adaptive.
3. **Fully adaptive adversary:** The adversary sees everything including current randomization before choosing y_t . Against this adversary, no non-trivial guarantees are possible (adversary can always force maximum loss).

Most randomized online learning algorithms guarantee performance against **non-anticipating adversaries**. Deterministic algorithms automatically work against fully adaptive adversaries (since there's no randomization to hide).

Why the distinction matters

- Oblivious: Weakest, easiest to compete against
- Non-anticipating: Standard assumption for randomized algorithms
- Fully adaptive: Too strong—makes learning impossible for randomized algorithms

 Construct Your Own

Task: Design an adversarial sequence for a naive “always follow the best expert so far” strategy.

Setup: Two experts, $n = 2$, over $T = 8$ rounds. Binary predictions (0 or 1).

Requirements:

- Construct outcomes that force the naive algorithm to make at least 4 mistakes
- Show that one expert makes at most 4 mistakes
- The algorithm switches experts at least twice

Hint: Make the “current best” expert wrong in the next round, forcing a switch.

Key Differences from Batch Learning:

Batch Learning	Online Learning
i.i.d. data assumption	No distributional assumptions
Fixed training set	Sequential decisions
Generalization error	Regret
Average-case analysis	Worst-case analysis
Offline optimization	Adaptive updates

✓ **Checkpoint**

Checkpoint 1: In the online learning protocol, why do we assume the adversary is “oblivious” (chooses losses before seeing our current randomization)?

Answer: Without this assumption, the adversary could see our random coin flip and always choose the worst outcome for us, making learning impossible. The oblivious adversary assumption is necessary for any meaningful guarantees while still allowing worst-case analysis.

The Prediction with Expert Advice Problem

Definition: Expert Advice Setting

- n experts: $E_1, E_2, \dots, E_n$
- T rounds of predictions
- Each round t :
 - Each expert i makes a prediction/recommendation
 - Learner chooses which expert(s) to follow
 - True outcome is revealed
 - Learner and experts incur losses

Definition: Loss Notation

We use two related notations for losses:

- **Loss function:** $\ell_t : \mathcal{A} \rightarrow \mathbb{R}$ is the loss function at time t , mapping actions to loss values. For expert i choosing action a_i^t , the loss is $\ell_{t(a_i^t)}$.
- **Loss values:** $m_{t(i)} = \ell_{t(a_i^t)}$ is the realized loss incurred by expert i in round t . This is a scalar value.

For the expert problem with binary outcomes:

- $m_{t(i)} = \mathbb{1}_{\text{expert } i \text{ makes mistake in round } t} \in \{0, 1\}$

For general losses:

- $m_{t(i)} \in [0, 1]$ (normalized/bounded losses)

Throughout this week, we primarily use $m_{t(i)}$ for concrete loss values.

Example: Stock Market Prediction

- $n = 100$ financial analysts
- Each day, each analyst predicts: “stock price will increase” or “decrease”
- You must decide: which analyst(s) to follow?
- At day’s end, the true price change is revealed
- Goal: minimize wrong predictions over T days

Regret measures how much worse the learner performs compared to the best single expert in hindsight. It’s the price of not knowing which expert is best ahead of time.

Regret: The Right Performance Measure

Why regret? In the i.i.d. setting, we compared to the Bayes-optimal classifier. In the adversarial setting, there’s no “optimal” strategy—the best we can hope for is to compete with the best fixed strategy in hindsight. Regret measures this gap.

Concrete example first: Suppose you’re choosing between 3 financial advisors over 100 days. You follow a strategy and lose \$1000 total. The best advisor in hindsight lost only \$600. Your regret is $1000 - 600 = 400$. Regret measures the cost of not knowing which advisor was best from the start.

Definition: Regret

The **regret** after T rounds is:

$$R_T = \sum_{t=1}^T \ell_t(\text{learner}) - \min_{i \in [n]} \sum_{t=1}^T \ell_t(\text{expert } i) \quad (1)$$

where $\ell_t(\cdot)$ denotes the **loss function** in round t (a function that assigns a numerical loss to each possible prediction—lower is better).

Interpretation:

- Regret = (learner’s total loss) – (best expert’s total loss)
- Measures performance relative to best expert in hindsight
- Goal: **sublinear regret** $R_T = o(T)$, meaning regret grows slower than T , which implies average regret vanishes: $\frac{R_T}{T} \rightarrow 0$

Example: If $R_T = \sqrt{T}$, then after $T = 10000$ rounds, regret is about 100. This means we’re only 100 losses behind the best expert—an average of $\frac{100}{10000} = 0.01$ loss per round. As $T \rightarrow \infty$, this average gap goes to zero.

Backbone: The key insight is that sublinear regret is achievable through multiplicative weight updates. Even though we don’t know which expert is best, we can learn to approximate their performance by exponentially downweighting bad experts.

Pause and Think

Question: Why do we compare to the best single expert, rather than trying to combine the “best of all experts”?

Hint: Consider two experts: one always predicts “increase”, the other always predicts “decrease”. One is always right, but neither is doing anything intelligent.

Answer:

If we tried to be better than both experts, we’d need to predict correctly every time—impossible without knowledge of the future. Competing with the best single expert is the right balance between ambitious and achievable.

✓ Checkpoint

Checkpoint 2: If an algorithm achieves $R_T = O(\sqrt{T})$, what is the average per-round regret $\frac{R_T}{T}$ as $T \rightarrow \infty$?

Answer: $\frac{R_T}{T} = \frac{O(\sqrt{T})}{T} = O\left(\frac{1}{\sqrt{T}}\right) \rightarrow 0$. The average regret vanishes, meaning asymptotically we perform as well as the best expert.

Types of Regret**Definition: External Regret**

Compares learner’s performance to the best fixed action:

$$R_T^{\text{ext}} = \sum_{t=1}^T \ell_t(a_t) - \min_{a \in \mathcal{A}} \sum_{t=1}^T \ell_t(a) \quad (2)$$

This is the standard notion we focus on. It asks: “How much worse did we do than if we had picked the single best action for all rounds?”

Remark

Other notions exist:

- **Internal regret:** Compares to best modification of learner’s own strategy (e.g., “what if we had switched from action i to action j every time we played i ?”)
- **Swap regret:** Compares to best strategy that swaps actions conditionally

These are more refined but beyond our scope. External regret is the most common and natural benchmark.

Adversarial vs. Stochastic Settings

Adversarial	Stochastic	The adversarial setting is more conservative but also more robust. Algorithms that work in the adversarial setting automatically work in the stochastic setting.
Losses chosen adversarially	Losses drawn from	
Worst-case analysis	Average-case analysis	
No assumptions on losses	i.i.d. assumption	
Regret: $O(\sqrt{T})$	$O(\log T)$ possible with stronger assumptions	
More robust	Better rates with assumptions	

Warm-up: The Perfect Expert Case

Before tackling the general problem, let's start with a simplified scenario that builds intuition.

Transition: We've seen that the adversarial setting makes learning hard—the environment can actively work against us. To build understanding gradually, we first consider an easier case where one expert is perfect. This will introduce the key idea of **eliminating** bad hypotheses, which we'll later soften to **downweighting** them.

Motivation: Sometimes the best way to understand a complex algorithm is to first solve a simpler version. The “perfect expert” assumption is unrealistic, but the Halving Algorithm that solves it introduces the key idea: eliminate bad hypotheses aggressively.

Definition: Perfect Expert Assumption

Assume there exists at least one expert who **never makes a mistake** throughout all T rounds.

Thought Experiment

Thought experiment: Imagine you have 1000 fortune tellers predicting coin flips. You know exactly one of them has genuine psychic powers (never wrong), but the rest are frauds (random guessing).

- After 1 wrong prediction, can someone be the psychic? **No—eliminate them.**
- After 10 rounds, roughly how many candidates remain? **About $\frac{1000}{2^{10}} \approx 1$**
- How many mistakes can you make using majority vote? **At most $\log_2 1000 \approx 10$**

This intuition leads directly to the Halving Algorithm.

The Halving Algorithm

Definition: Halving Algorithm

- Maintain set $S_t \subseteq \{1, \dots, n\}$ of experts that haven't made mistakes
- Initially: $S_1 = \{1, 2, \dots, n\}$
- Each round t :
 1. Predict by **majority vote** among experts in S_t
 2. Remove experts that made mistakes from S_t

Theorem: Halving Algorithm Mistake Bound

If a perfect expert exists, the Halving Algorithm makes at most $\log_2 n$ mistakes.

Proof.

- Initially: $|S_1| = n$
- When algorithm makes a mistake, at least half of S_t predicted incorrectly
- These experts are removed: $|S_{t+1}| \leq \frac{|S_t|}{2}$
- After m mistakes: $|S_{T+1}| \leq \frac{n}{2^m}$
- Perfect expert never removed: $|S_{T+1}| \geq 1$
- Therefore: $\frac{n}{2^m} \geq 1 \implies m \leq \log_2 n$

■

Example: Concrete Example

With $n = 8$ experts:

- Round 1: Algorithm makes mistake, removes 4 experts $\rightarrow$ 4 remain
- Round 2: Algorithm makes mistake, removes 2 experts $\rightarrow$ 2 remain
- Round 3: Algorithm makes mistake, removes 1 expert $\rightarrow$ 1 remains (the perfect expert)
- Total mistakes: $3 = \log_2 8$

Wrong Path: Why Halving Fails in Practice

The Halving Algorithm has a fatal flaw: it requires a perfect expert to exist. In reality:

- All experts make mistakes
- Set S_t eventually becomes empty
- Algorithm has no one left to follow

We need an algorithm that gracefully handles imperfect experts.

Key Takeaway

The Halving Algorithm achieves logarithmic mistake bound when a perfect expert exists, but fails completely without one. The key insight: we can **reduce weights** instead of eliminating experts entirely. This leads to the Weighted Majority Algorithm.

→ Week 11 Preview: Neural Network Training and SGD

Week 11 will study stochastic gradient descent (SGD) for training neural networks. SGD is exactly online gradient descent applied to mini-batches! The $O(\sqrt{T})$ regret bounds we derive this week explain why SGD converges even with stochastic gradients, and provide foundations for understanding adaptive optimizers like Adam (which use multiplicative weight-like updates on gradient moments).

 Confidence Check

Self-Assessment: Before moving to the general case, check your understanding:

Rate your confidence (1-5):

- ☐ I can explain why Halving makes at most $\log_2 n$ mistakes
- ☐ I understand why Halving fails without a perfect expert
- ☐ I can construct an adversarial sequence that breaks “follow the current best”
- ☐ I see the connection between eliminating experts and hypothesis class pruning

If 3 or below on any item: Review the Halving proof and the thought experiment above.

The Weighted Majority Algorithm

Instead of eliminating experts who make mistakes, we can **reduce their influence** by decreasing their weights. This gracefully handles the realistic case where all experts make mistakes.

Backbone: Here's where the multiplicative weight update principle emerges. Instead of binary elimination (weight 1 or 0), we use exponential decay: $w \leftarrow (1 - \eta)w$. This soft elimination is the key to achieving sublinear regret without perfect experts.

Deterministic Weighted Majority

Motivation: The transition from Halving to Weighted Majority is profound. Halving asks “Who’s perfect?” and fails when no one is. Weighted Majority asks “Who’s better on average?” and gracefully tracks relative performance through weight decay.

Productive Struggle

Challenge: Before reading the algorithm, try to design your own weight-based expert selection scheme.

Setup: $n = 3$ experts, binary predictions, no perfect expert assumption.

Your task (spend 10-15 minutes):

1. How should you initialize weights?
2. What rule determines your prediction from weights?
3. When an expert is wrong, how should their weight change?
4. Should you ever increase weights?

Guiding questions:

- Should weight updates be additive ($w \leftarrow w - c$) or multiplicative ($w \leftarrow \alpha w$)?
- What happens if you decrease by a constant vs. a fraction?
- How do you ensure weights never become negative?

After struggling: Compare your design to the Weighted Majority Algorithm below. What’s similar? What’s different? Why might multiplicative updates be better than additive?

Definition: Weighted Majority Algorithm**Parameters:** Learning rate $\eta \in (0, \frac{1}{2}]$ **Initialization:** $w_1(i) = 1$ for all experts $i \in [n]$ **For each round $t = 1, \dots, T$:**

1. **Predict:** Follow weighted majority

$$\text{Predict 0 if } \sum_{i: \text{predict 0}} w_t(i) \geq \sum_{i: \text{predict 1}} w_t(i), \text{ else predict 1} \quad (3)$$

2. **Observe:** True outcome and which experts were correct/wrong

3. **Update:** For each expert i :

$$w_{t+1}(i) = \begin{cases} (1 - \eta)w_t(i) & \text{if expert } i \text{ was wrong} \\ w_t(i) & \text{if expert } i \text{ was correct} \end{cases} \quad (4)$$

The multiplicative update $(1 - \eta)$ is the key innovation. Experts who make mistakes see their weights decay exponentially over time.

Intuition:

- Start with equal trust in all experts
- When an expert is wrong, reduce trust by factor $(1 - \eta)$
- Prediction is weighted majority vote
- Over time, better experts accumulate more weight

Example: Weight EvolutionConsider $n = 3$ experts with $\eta = 0.1$:

Round	Expert 1	Expert 2	Expert 3	Mistakes
1	1.000	1.000	1.000	E_1 wrong
2	0.900	1.000	1.000	E_2 wrong
3	0.900	0.900	1.000	E_1, E_2 wrong
4	0.810	0.810	1.000	E_3 wrong
5	0.810	0.810	0.900	—

Expert 3 (fewest mistakes) has highest weight.

✓ Checkpoint**Checkpoint 3:** In the weight evolution example above, after round 5, what is the ratio of Expert 3's weight to Expert 1's weight?**Answer:** $\frac{0.900}{0.810} = 1.11\dots$ Expert 3 has about 11% more influence. After more rounds, this ratio grows exponentially: if Expert 3 makes k fewer mistakes, the ratio is $\left(\frac{1}{1-\eta}\right)^k \approx (1.11)^k$.**Regret Bound via Potential Function Analysis**

Motivation: How do we prove the algorithm works? The **potential function** is a technique that tracks a carefully chosen quantity over time to bound the regret. This is a classic approach: track a quantity ($\Phi_t = \sum_i w_{t(i)}$, the total weight), bound it from above (using algorithm's mistakes) and below (using best expert's mistakes), then extract the regret inequality by combining bounds.

Deterministic = Adaptive-robust

Since Weighted Majority is deterministic, it works against fully adaptive adversaries. The adversary can see all past actions and current weights, but this doesn't help—the algorithm's behavior is deterministic given the history.

Proof Template: Potential Function Method

This proof pattern appears throughout online learning and amortized analysis:

1. **Define potential:** $\Phi_t = \sum_i w_{t(i)}$ (total weight)
2. **Initial/final values:** $\Phi_1 = n$, $\Phi_{T+1} \geq w_{T+1}(i^*)$ for best expert
3. **Upper bound:** Relate $\frac{\Phi_{t+1}}{\Phi_t}$ to algorithm's loss
4. **Lower bound:** Relate Φ_{T+1} to best expert's loss
5. **Combine:** Sandwich Φ_{T+1} between upper and lower bounds
6. **Extract regret:** Solve for algorithm's loss in terms of best expert's loss

When to use: Weight-based algorithms, multiplicative updates, amortized analysis

Theorem: Weighted Majority Mistake Bound

Assume there are n experts making binary predictions.

Against any adversary (including fully adaptive, since the algorithm is deterministic), the Weighted Majority algorithm with learning rate $\eta \in (0, 1)$ satisfies:

Let M be the total number of mistakes made by the algorithm over T rounds, and M^* be the number of mistakes made by the best expert. Then:

$$M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta} \quad (5)$$

Optimal tuning: Choosing $\eta = \sqrt{\frac{\ln n}{M^*}}$ (requires knowing M^* in advance) gives:

$$M \leq 2\sqrt{M^* \ln n} + O(\sqrt{\ln n}) \quad (6)$$

Proof Strategy: We'll use the **potential function method**:

1. Define potential $\Phi_t = \sum_i w_t(i)$
2. Derive upper bound on Φ_{T+1} using algorithm's mistakes
3. Derive lower bound on Φ_{T+1} using best expert's mistakes
4. Combine bounds to get regret inequality

Proof. Step 1: Define Potential Function

Let $\Phi_t = \sum_{i=1}^n w_t(i)$ be the total weight at round t .

- Initially: $\Phi_1 = n$ (all weights start at 1)
- Finally: $\Phi_{T+1} \geq w_{T+1}(i^*)$ for best expert i^*

Step 2: Upper Bound on Φ_{T+1}

Consider what happens when the algorithm makes a mistake in round t :

- Algorithm predicted by weighted majority
- At least half the total weight was on wrong experts
- These weights get multiplied by $(1 - \eta)$
- Other weights stay the same

Therefore:

$$\begin{aligned}
 \Phi_{t+1} &= \sum_{\text{correct}} w_t(i) + \sum_{\text{wrong}} (1 - \eta)w_t(i) \\
 &= \sum_{\text{correct}} w_t(i) + (1 - \eta) \sum_{\text{wrong}} w_t(i) \\
 &\leq \frac{1}{2}\Phi_t + (1 - \eta) \cdot \frac{1}{2}\Phi_t \\
 &= \Phi_t \left(\frac{1}{2} + \frac{1}{2}(1 - \eta) \right) \\
 &= \Phi_t \left(1 - \frac{\eta}{2} \right)
 \end{aligned} \tag{7}$$

The key inequality $\sum_{\text{wrong}} w_t(i) \geq \frac{\Phi_t}{2}$ holds because when the algorithm makes a mistake, the weighted majority (which is at least half the total weight) must have predicted wrong.

After M mistakes:

$$\Phi_{T+1} \leq n \left(1 - \frac{\eta}{2} \right)^M \tag{8}$$

Step 3: Lower Bound on Φ_{T+1}

Let i^* be the best expert (fewest mistakes M^*). Then:

$$\Phi_{T+1} = \sum_{i=1}^n w_{T+1}(i) \geq w_{T+1}(i^*) = (1 - \eta)^{M^*} \tag{9}$$

The last equality holds because expert i^* started with weight 1 and was multiplied by $(1 - \eta)$ exactly M^* times.

Step 4: Combine Bounds

From Steps 2 and 3:

$$(1 - \eta)^{M^*} \leq \Phi_{T+1} \leq n \left(1 - \frac{\eta}{2} \right)^M \tag{10}$$

Taking logarithms:

$$M^* \ln(1 - \eta) \leq \ln n + M \ln \left(1 - \frac{\eta}{2} \right) \tag{11}$$

Step 5: Apply Logarithmic Inequalities

We need the following bounds (see lemma below):

- $\ln(1 - \eta) \geq -\eta - \eta^2$ for $\eta \in (0, \frac{1}{2}]$
- $\ln(1 - \frac{\eta}{2}) \leq -\frac{\eta}{2}$

Substituting:

$$-M^*(\eta + \eta^2) \leq M^* \ln(1 - \eta) \leq \ln n + M \ln\left(1 - \frac{\eta}{2}\right) \leq \ln n - M \frac{\eta}{2} \quad (12)$$

Therefore:

$$-M^*(\eta + \eta^2) \leq \ln n - M \frac{\eta}{2} \quad (13)$$

Rearranging:

$$M \frac{\eta}{2} \leq M^*(\eta + \eta^2) + \ln n \quad (14)$$

$$M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta} \quad (15)$$

■

✓ Checkpoint

Checkpoint 4: In the potential function proof, why does $\Phi_{t+1} \leq \Phi_{t(1-\frac{\eta}{2})}$ when the algorithm makes a mistake?

Answer: When the algorithm predicts wrong, at least half the weight ($\geq \frac{\Phi_t}{2}$) was on wrong experts. Those weights get multiplied by $(1 - \eta)$, while correct weights stay the same: $\Phi_{t+1} \leq \frac{\Phi_t}{2} + \frac{\Phi_t}{2}(1 - \eta) = \Phi_{t(1-\frac{\eta}{2})}$.

Lemma: Logarithmic Bounds

For $\eta \in (0, \frac{1}{2}]$:

$$-\eta - \eta^2 \leq \ln(1 - \eta) \leq -\eta \quad (16)$$

Proof. Upper bound: The Taylor series expansion of $\ln(1 - \eta)$ around $\eta = 0$ is:

$$\ln(1 - \eta) = -\sum_{k=1}^{\infty} \frac{\eta^k}{k} = -\eta - \frac{\eta^2}{2} - \frac{\eta^3}{3} - \dots \quad (17)$$

All terms are negative (since $\eta > 0$). Taking only the first term gives an upper bound:

$$\ln(1 - \eta) \leq -\eta \quad (18)$$

Lower bound: Consider $f(\eta) = \ln(1 - \eta) + \eta + \eta^2$. Then:

$$f'(\eta) = -\frac{1}{1 - \eta} + 1 + 2\eta = \frac{\eta(1 - 2\eta)}{1 - \eta} \geq 0 \quad (19)$$

for $\eta \in (0, \frac{1}{2}]$. Since $f(0) = 0$ and f is increasing, $f(\eta) \geq 0$, giving $\ln(1 - \eta) \geq -\eta - \eta^2$. ■

Proof Template: Logarithmic Inequality Arguments

This technique appears throughout regret analysis:

1. **Identify exponential/logarithmic terms:** Upper bound has $(1 - \frac{\eta}{2})^M$, lower bound has $(1 - \eta)^{M^*}$
2. **Take logarithms:** Convert products to sums
3. **Apply Taylor bounds:** Use $\ln(1 - x) \approx -x$ for small x , with error terms
4. **Choose appropriate bounds:** Upper bound needs $\ln(1 - x) \leq -x$, lower bound needs $\ln(1 - x) \geq -x - x^2$
5. **Solve for regret:** Combine and rearrange

When to use: Multiplicative weight updates, exponential decay, geometric series

Key insight: The gap between $\ln(1 - x) \leq -x$ and $\ln(1 - x) \geq -x - x^2$ determines the regret bound's quality. Tighter approximations (more Taylor terms) give better constants.

Optimizing the Learning Rate

The bound $M \leq 2(1 + \eta)M^* + \frac{2\ln n}{\eta}$ has two terms:

- **Multiplicative term:** $2(1 + \eta)M^*$ — grows with η
- **Additive term:** $\frac{2\ln n}{\eta}$ — shrinks with η

← From Week 4: Bias-Variance Tradeoff

This tradeoff mirrors Week 4's bias-variance decomposition! Large $\eta \rightarrow$ aggressive weight updates $\rightarrow$ low “bias” (track best expert quickly) but high “variance” (sensitive to noise). Small $\eta \rightarrow$ conservative updates $\rightarrow$ high bias (slow to identify best) but low variance (stable). The optimal η balances these like optimal model complexity balances bias-variance.

Pause and Think

Question: How should we choose η to minimize the bound?

Hint: Balance the two terms: set $2\eta M^* \approx \frac{2\ln n}{\eta}$

Answer:

Optimal choice: $\eta = \sqrt{\frac{\ln n}{M^*}}$, giving $M \leq 2M^* + 4\sqrt{M^* \ln n}$. However, this requires knowing M^* in advance, which we don't!

Remark

In practice, we can:

1. Use a fixed small η (e.g., $\eta = 0.1$)
2. Use the **doubling trick** to handle unknown T
3. Set $\eta = \sqrt{\frac{\ln n}{T}}$ if T is known

This lower bound shows the factor of 2 in our theorem is tight for deterministic algorithms. To do better, we need randomization!

The Factor of 2 is Unavoidable

Theorem: Lower Bound for Deterministic Algorithms

Any deterministic algorithm must have $M \geq 2M^*$ in the worst case.

Proof. Consider $n = 2$ experts:

- Expert 1 always predicts 0
- Expert 2 always predicts 1

Adversary chooses outcomes to make algorithm wrong:

- If algorithm predicts 0, outcome is 1
- If algorithm predicts 1, outcome is 0

For even T , the algorithm makes T mistakes while the best expert makes $\frac{T}{2}$ mistakes. Ratio: $\frac{M}{M^*} = 2$; for odd T , the ratio approaches 2 up to rounding. ■

Key Takeaway

The Weighted Majority Algorithm achieves $M \leq 2(1 + \eta)M^* + O(\log n)$ mistakes. The factor of 2 is unavoidable for deterministic algorithms, but the logarithmic dependence on n is excellent. The potential function method provides a clean proof technique applicable to many online learning algorithms.

 Construct Your Own

Task: Design a scenario where the deterministic Weighted Majority Algorithm is forced to make close to $2M^*$ mistakes.

Requirements:

- Two experts ($n = 2$)
- Construct a loss sequence over $T = 100$ rounds
- Show that the algorithm makes ≈ 100 mistakes while best expert makes ≈ 50
- Explain why the factor of 2 is tight

Hint: Think about the two-expert lower bound proof. What if one expert always predicts 0, the other always predicts 1, and the adversary chooses outcomes to make the weighted majority wrong?

Randomized Weighted Majority

We've seen that the deterministic Weighted Majority algorithm achieves a factor-of-2 approximation to the best expert, and we've proven this factor is unavoidable for deterministic algorithms. This raises a natural question: **can we do better?**

The key insight: To break the factor-of-2 barrier (the unavoidable constant 2 in the deterministic algorithm's bound), we turn to randomization. Instead of deterministically following the weighted majority, we'll sample an expert proportionally to weights. This seemingly minor change—predict by sampling instead of voting—improves the constant from 2 to 1 in the regret bound.

Why randomization helps: When two experts have equal weight, a deterministic algorithm must commit to one prediction, allowing an adversary to make that prediction wrong. A randomized algorithm samples 50-50 and is wrong only half the time on average—the adversary cannot exploit deterministic patterns.

All guarantees in this subsection are in expectation over the learner's randomness.

Progress on backbone: Randomization is the key to optimal regret. This principle—using randomness to prevent adversarial exploitation—appears throughout computer science, from cryptography to algorithm design.

← From Week 3: Concentration Inequalities

In Week 3, we studied how randomness concentrates: independent random variables converge to their expectation. Here, randomization serves a different purpose—it prevents adversarial exploitation. The adversary can't force us to always be wrong when our predictions are random, even if they know our algorithm!

The Randomized Algorithm

Motivation: Why does randomization help? Consider two experts with equal weights (50-50). A deterministic algorithm must commit to one prediction, and an adversary can make that prediction wrong. A randomized algorithm is wrong only 50% of the time on average—the adversary can't exploit deterministic patterns.

Definition: Randomized Weighted Majority**Parameters:** Learning rate $\eta \in (0, \frac{1}{2}]$ **Initialization:** $w_1(i) = 1$ for all experts $i \in [n]$ **For each round $t = 1, \dots, T$:**

1. **Predict:** Sample expert i with probability

$$p_t(i) = \frac{w_t(i)}{\Phi_t} \quad \text{where} \quad \Phi_t = \sum_{j=1}^n w_t(j) \quad (20)$$

2. **Observe:** True outcome and losses

3. **Update:** For each expert i :

$$w_{t+1}(i) = \begin{cases} (1 - \eta)w_t(i) & \text{if expert } i \text{ was wrong} \\ w_t(i) & \text{if expert } i \text{ was correct} \end{cases} \quad (21)$$

Randomization allows the algorithm to “hedge its bets” when uncertain, avoiding the worst-case scenarios that plague deterministic algorithms.

Key Difference: Instead of following the majority deterministically, we sample proportionally to weights. If weights are balanced, we’re uncertain and make a 50-50 guess.

 Construct Your Own**Task:** Simulate the randomized algorithm on a concrete sequence.**Setup:** $n = 2$ experts, $\eta = 0.5$, $T = 4$ rounds

Loss sequence:

- Round 1: Expert 1 wrong (outcome = 1, Expert 1 predicted 0)
- Round 2: Expert 2 wrong
- Round 3: Expert 1 wrong
- Round 4: Expert 2 wrong

Your tasks:

1. Compute weights w_1, w_2, w_3, w_4, w_5
2. Compute probabilities $p_{t(1)}, p_{t(2)}$ for each round
3. Calculate expected number of mistakes: $\sum_{t=1}^4 p_{t(i_{\text{wrong}})}$
4. Compare to deterministic Weighted Majority (what would it predict each round?)

Reflection: Which algorithm performs better on this sequence?**Improved Regret Bound**

Theorem: Randomized Weighted Majority Bound

Assume there are n experts making binary predictions, and learn

Against a non-anticipating adversary, the Randomized Weighted Majority Bound satisfies:

Let M be the random variable representing the number of mistakes over T rounds, and M^* be the number of mistakes made by the best expert. The expected number of mistakes (expectation over the learner's randomization) is:

$$\mathbb{E}[M] \leq (1 + \eta)M^* + \frac{\ln n}{\eta} \quad (22)$$

Optimal tuning: Choosing $\eta = \sqrt{\frac{\ln n}{M^*}}$ gives:

$$\mathbb{E}[M] \leq \sqrt{M^* \ln n} + O(\sqrt{\ln n}) \quad (23)$$

Non-anticipating adversary

The adversary can see all past predictions and weights, but cannot see the current random sample before choosing the outcome. This prevents the adversary from always targeting the randomly selected expert.

Compare to deterministic: in expectation, the leading factor improves from $2(1 + \eta)$ to $(1 + \eta)$, and the additive term improves from $\frac{2 \ln n}{\eta}$ to $\frac{\ln n}{\eta}$.

Proof. Step 1: Expected Mistakes

Let frac_t = weighted fraction of wrong experts in round t (we use “frac” to distinguish from loss function notation):

$$\text{frac}_t = \frac{\sum_{i: \text{wrong}} w_t(i)}{\Phi_t} \quad (24)$$

The probability of making a mistake in round t is exactly frac_t (we sample proportionally to weights). Therefore:

$$\mathbb{E}[M] = \sum_{t=1}^T \text{frac}_t \quad (25)$$

Step 2: Potential Function

As before, $\Phi_t = \sum_i w_t(i)$ with $\Phi_1 = n$.

Lower bound (same as before):

$$\Phi_{T+1} \geq w_{T+1}(i^*) = (1 - \eta)^{M^*} \quad (26)$$

Step 3: Upper Bound on Potential

At time t , the learner samples expert $i_t \sim p_t$ (with probability $p_{t(i)}$ for each expert i) and suffers loss $m_{t(i_t)}$.

The learner's **expected loss** at time t (expectation over the randomization $i_t \sim p_t$) is:

$$\mathbb{E}_{i_t \sim p_t} [m_{t(i_t)}] = \sum_{i=1}^n p_{t(i)} m_{t(i)} = \langle p_t, m_t \rangle = \text{frac}_t \quad (27)$$

where frac_t is the weighted fraction of wrong experts in round t (we use “frac” to distinguish from loss function notation).

In round t , fraction frac_t of the weight is on wrong experts:

$$\begin{aligned}
\Phi_{t+1} &= \sum_{\text{correct}} w_t(i) + \sum_{\text{wrong}} (1 - \eta)w_t(i) \\
&= (1 - \text{frac}_t)\Phi_t + \text{frac}_t(1 - \eta)\Phi_t \\
&= \Phi_t(1 - \eta \cdot \text{frac}_t) \\
&\leq \Phi_t e^{-\eta \text{frac}_t}
\end{aligned} \tag{28}$$

The last step uses $1 - x \leq e^{-x}$ for all x .

Telescoping over all rounds:

$$\Phi_{T+1} \leq \Phi_1 \exp\left(-\eta \sum_{t=1}^T \text{frac}_t\right) = n \exp(-\eta \mathbb{E}[M]) \tag{29}$$

where the last equality uses $\mathbb{E}[M] = \sum_t \text{frac}_t$ (the expected mistakes equal the sum of fractions of wrong weight).

Step 4: Combine Bounds

$$(1 - \eta)^{M^*} \leq \Phi_{T+1} \leq n \exp(-\eta \mathbb{E}[M]) \tag{30}$$

Taking logarithms:

$$M^* \ln(1 - \eta) \leq \ln n - \eta \mathbb{E}[M] \tag{31}$$

Using $\ln(1 - \eta) \geq -\eta - \eta^2$:

$$-M^*(\eta + \eta^2) \leq \ln n - \eta \mathbb{E}[M] \tag{32}$$

Solving for $\mathbb{E}[M]$:

$$\eta \mathbb{E}[M] \leq M^*(\eta + \eta^2) + \ln n \tag{33}$$

$$\mathbb{E}[M] \leq (1 + \eta)M^* + \frac{\ln n}{\eta} \tag{34}$$

■

🏗 Scaffolding Derivation (3 steps)

Let's build the MWU regret bound step-by-step with decreasing support.

Step 1 (Full support): Prove the key inequality $1 - x \leq e^{-x}$ for $x \geq 0$.

Proof: Consider $f(x) = e^{-x} + x - 1$. Then $f'(x) = -e^{-x} + 1$.

- For $x > 0$: $e^{-x} < 1$, so $f'(x) > 0$ (increasing)
- For $x = 0$: $f(0) = 0$
- Therefore: $f(x) \geq 0$ for all $x \geq 0$, giving $1 - x \leq e^{-x}$

Step 2 (Partial support): Use this inequality to bound the potential function.

In round t , fraction f_t of weight is wrong. The potential evolves:

$$\Phi_{t+1} = \Phi_t(1 - \eta f_t) \leq \Phi_t e^{-\eta f_t} \quad (35)$$

Your turn: Telescope this over all T rounds to get $\Phi_{T+1} \leq n \exp(-\eta \sum_{t=1}^T f_t)$.

Hint: $\Phi_{t+1} \leq \Phi_t e^{-\eta f_t} \Rightarrow \frac{\Phi_{t+1}}{\Phi_t} \leq e^{-\eta f_t}$. Multiply over all t .

Step 3 (Minimal support): Combine with the lower bound.

We have:

- Upper: $\Phi_{T+1} \leq n \exp(-\eta \mathbb{E}[M])$ (since $\mathbb{E}[M] = \sum_t f_t$)
- Lower: $\Phi_{T+1} \geq (1 - \eta)^{M^*}$

Your turn: Take logarithms of both sides and solve for $\mathbb{E}[M]$ in terms of M^* .

Challenge: What approximation do you need for $\ln(1 - \eta)$?

Pause and Think

Question: Why does randomization help? What's the intuition?

Hint: In the two-expert example (one always predicts 0, one always predicts 1), a deterministic algorithm can be forced to always be wrong.

Answer:

With randomization, when weights are balanced (50-50), we make a random guess and are wrong only half the time on average. Randomization prevents the adversary from exploiting our deterministic strategy.

✓ Checkpoint

Checkpoint 5: Why does the randomized algorithm achieve expected regret $\mathbb{E}[M] \leq (1 + \eta)M^* + \frac{\ln n}{\eta}$ while deterministic achieves $M \leq 2(1 + \eta)M^* + \frac{2 \ln n}{\eta}$?

Answer: The key difference is in the upper bound on Φ_{t+1} . Deterministic: at least **half** the weight is wrong when we make a mistake (factor of 2). Randomized: **exactly** fraction f_t of weight is wrong in expectation (no factor of 2). Using $1 - x \leq e^{-x}$ gives the tighter bound.

Optimal Learning Rate

Setting $\eta = \min\left(\frac{1}{2}, \sqrt{\frac{\ln n}{M^*}}\right)$ (if M^* known) gives, in the usual nontrivial regime:

$$\mathbb{E}[M] \leq M^* + 2\sqrt{M^* \ln n} \quad (36)$$

The leading term is now M^* (not $2M^*$), with only a lower-order $\sqrt{M^* \ln n}$ term!

→ Week 11 Preview: Adam and Adaptive Learning Rates

Week 11's deep learning section covers adaptive optimizers like Adam, which adjust learning rates per parameter. Adam uses exponential moving averages of gradients (first moment m_t) and squared gradients (second moment v_t), then updates: $\theta \leftarrow \theta - \eta \frac{m_t}{\sqrt{v_t}}$. This is MWU-inspired: the $\sqrt{v_t}$ denominator adaptively scales η based on gradient history, just like we optimize η based on M^* !

Multiplicative Weights Update (MWU)

We first present the clean $[0, 1]$ loss case, which already covers mistake indicators. Signed losses in $[-1, 1]$ are usually handled by the exponential-weights update $w_i \leftarrow w_i \exp(-\eta m_t(i))$; negative losses then increase weights and represent gains.

We've mastered the expert advice problem with binary outcomes raises a natural question: what if losses aren't binary? What if we have (continuous values), or optimizing click-through rates (probabilities), or errors (real numbers)?

The generalization challenge: So far, we've considered binary predictions (correct/wrong). Now we generalize to **real-valued losses** (losses that can be any number in a range, not just 0 or 1). The Randomized Weighted Majority algorithm handles mistakes (loss 0 or 1). But many real problems have continuous losses: portfolio returns, prediction errors, click-through rates. We need a framework that handles arbitrary losses in $[0, 1]$ while preserving the $O(\sqrt{T})$ regret guarantee.

The beautiful discovery: The Multiplicative Weights Update framework generalizes everything we've learned to arbitrary losses in $[0, 1]$. Even more remarkably, it connects to AdaBoost from last week—revealing that boosting and online learning are two faces of the same coin.

Progress on backbone: MWU is one of the most fundamental algorithms in computer science. It appears in optimization, game theory, machine learning, and even approximation algorithms. Understanding it opens doors to vast areas of theory and practice.

Backbone: This is the full power of multiplicative weights. Instead of binary “mistake or not”, experts incur losses in $[0, 1]$, and we update weights continuously: $w \leftarrow w(1 - \eta \ell)$. This single formula unifies expert advice, online optimization, game theory, and even AdaBoost—all are instances of exponential reweighting.

General Framework

Motivation: Binary losses (0/1) are too restrictive. In portfolio optimization, losses are continuous. In online advertising, click-through rates are between 0 and 1. We need a framework that handles arbitrary losses while preserving the $O(\sqrt{T})$ regret guarantee.

Concrete example: Consider online portfolio management. Each day t , you allocate your budget across n stocks (experts). Stock i returns $r_{t(i)} \in [-1, 1]$ (negative means loss). Your loss for choosing allocation p_t is $m_{t(i)} = -r_{t(i)}$ normalized to $[0, 1]$. Your expected loss is the weighted average: $\sum_i p_{t(i)} m_{t(i)}$. This is continuous, not binary.

Definition: MWU Setting

- n experts
- T rounds
- Loss vector $m_t = (m_t(1), \dots, m_t(n))$ where $m_t(i) \in [0, 1]$ is the loss incurred by expert i in round t
- Learner chooses distribution p_t over experts (i.e., p_t is a probability vector with $p_{t(i)} \geq 0$ and $\sum_i p_{t(i)} = 1$)
- Learner's expected loss: $\langle p_t, m_t \rangle = \sum_i p_t(i) m_t(i)$ (the **inner product** or dot product, which gives the weighted average loss)

Key Generalization: Instead of “mistake” or “no mistake”, each expert incurs a real-valued loss. The learner’s expected loss is the weighted average.

Theorem Landscape

This section presents MWU in increasing generality:

- **Algorithm 3.1 (MWU):** Multiplicative weight update rule with losses in $[0, 1]$
- **Theorem 3.1 (MWU Regret Bound):** $O\left(\eta T + \frac{\ln n}{\eta}\right)$ regret, optimal at $\eta = \sqrt{\frac{\ln n}{T}}$
- **Proof Strategy:** Same potential function method, but now with continuous losses

Relationship to previous results:

- When $m_{t(i)} \in \{0, 1\}$, MWU reduces to Randomized Weighted Majority
- The proof technique is identical, with binary mistakes generalized to continuous losses

Why this matters: Real-valued losses are essential for applications. Portfolio optimization, online advertising, and resource allocation all require continuous loss functions. MWU provides the algorithmic foundation.

Remark

The MWU theorem generalizes Randomized Weighted Majority:

Randomized WM: Binary losses $m_{t(i)} \in \{0, 1\}$ (mistakes), expected loss $\langle p_t, m_t \rangle = \mathbb{P}[\text{algorithm makes mistake}]$

MWU: General losses $m_{t(i)} \in [0, 1]$, expected loss $\langle p_t, m_t \rangle = \sum_i p_{t(i)} m_{t(i)}$

The proof technique is identical—exponential weight updates and logarithmic potential analysis.

Definition: Multiplicative Weights Update Algorithm

Parameters: Learning rate $\eta \in (0, \frac{1}{2}]$

Initialization: $w_1(i) = 1$ for all $i \in [n]$

For each round $t = 1, \dots, T$:

1. **Choose distribution:**

$$p_t(i) = \frac{w_t(i)}{\Phi_t} \quad \text{where} \quad \Phi_t = \sum_{j=1}^n w_t(j) \quad (37)$$

2. **Observe:** Loss vector $m_t = (m_t(1), \dots, m_t(n))$

3. **Update:**

$$w_{t+1}(i) = w_t(i) \cdot (1 - \eta m_t(i)) \quad (38)$$

Remark

When $m_t(i) \in \{0, 1\}$, this reduces to Randomized Weighted Major

- $m_t(i) = 0$ (correct): $w_{t+1}(i) = w_t(i)$
- $m_t(i) = 1$ (wrong): $w_{t+1}(i) = (1 - \eta)w_t(i)$

Why non-anticipating?

MWU uses randomization (sampling expert i with probability $p_{t(i)}$). Against a fully adaptive adversary that sees the sampled expert before choosing losses, no non-trivial guarantee is possible. The non-anticipating adversary can see past samples but not the current randomization.

MWU Regret Bound**Theorem: MWU Regret Bound**

Assume:

- n experts with losses $m_{t(i)}$ revealed in each round $t = 1, \dots, T$
- Loss values are bounded: $m_{t(i)} \in [0, 1]$ for all t, i
- Learning rate: $\eta \in (0, \frac{1}{2}]$

Against a non-anticipating adversary, the MWU algorithm achieves regret:

$$\text{Regret}_T = \sum_{t=1}^T \langle p_t, m_t \rangle - \min_{i \in [n]} \sum_{t=1}^T m_t(i) \leq \frac{\ln n}{\eta} + \eta T \quad (39)$$

Optimal tuning: Setting $\eta = \sqrt{\frac{\ln n}{T}}$ gives:

$$\text{Regret}_T \leq 2\sqrt{T \ln n} \quad (40)$$

This is optimal up to constant factors (lower bound: $\Omega(\sqrt{T \ln n})$).

Simplified Bound: Since $m_t(i) \leq 1$,

$$\sum_{t=1}^T \langle p_t, m_t \rangle - \min_i \sum_{t=1}^T m_t(i) \leq \eta T + \frac{\ln n}{\eta} \quad (41)$$

Optimal $\eta = \sqrt{\frac{\ln n}{T}}$ gives:

$$\text{Regret} \leq 2\sqrt{T \ln n} \quad (42)$$

This is **sublinear** in T : average regret $\rightarrow 0$ as $T \rightarrow \infty$.

Proof. Step 1: Potential Function

$\Phi_t = \sum_j w_t(j)$ with $\Phi_1 = n$ and $\Phi_{T+1} \geq w_{T+1}(i)$ for any expert i .

Step 2: Upper Bound on Φ_{T+1}

$$\begin{aligned} \Phi_{t+1} &= \sum_{j=1}^n w_t(j)(1 - \eta m_t(j)) \\ &= \Phi_t(1 - \eta \langle p_t, m_t \rangle) \\ &\leq \Phi_t \exp(-\eta \langle p_t, m_t \rangle) \end{aligned} \quad (43)$$

Telescoping:

$$\Phi_{T+1} \leq n \exp \left(-\eta \sum_{t=1}^T \langle p_t, m_t \rangle \right) \quad (44)$$

Step 3: Lower Bound on Φ_{T+1}

For any expert i with losses $m_t(i) \in [0, 1]$:

$$w_{T+1}(i) = w_1(i) \prod_{t=1}^T (1 - \eta m_t(i)) = \prod_{t=1}^T (1 - \eta m_t(i)) \quad (45)$$

where we used $w_1(i) = 1$ (initial weight).

Using the Taylor inequality $\ln(1 - \eta x) \geq -\eta x - \eta^2 x$ for $x \in [0, 1]$ and $\eta \in (0, \frac{1}{2}]$.

Applying this inequality with $x = m_{t(i)} \in [0, 1]$:

$$\begin{aligned} \ln w_{T+1}(i) &= \ln \left(\prod_{t=1}^T (1 - \eta m_{t(i)}) \right) \\ &= \sum_{t=1}^T \ln(1 - \eta m_{t(i)}) \\ &\geq \sum_{t=1}^T (-\eta m_{t(i)} - \eta^2 m_{t(i)}) \\ &= -\eta \sum_{t=1}^T m_{t(i)} - \eta^2 \sum_{t=1}^T m_{t(i)} \\ &= -(\eta + \eta^2) \sum_{t=1}^T m_{t(i)} \end{aligned} \quad (46)$$

where the first equality uses $\ln(\prod) = \sum \ln$, and the inequality applies the Taylor bound to each term.

Step 4: Combine

$$-(\eta) \sum_{t=1}^T \langle p_t, m_t \rangle + \ln n \geq \ln \Phi_{T+1} \geq \ln w_{T+1}(i) \quad (47)$$

$$\geq -(\eta + \eta^2) \sum_{t=1}^T m_t(i). \quad (48)$$

Rearranging and using $\eta \leq 1$ gives:

$$\sum_{t=1}^T \langle p_t, m_t \rangle \leq (1 + \eta) \sum_{t=1}^T m_t(i) + \frac{\ln n}{\eta}. \quad (49)$$

■

Key Takeaway

MWU achieves $O(\sqrt{T})$ regret against adversarial losses, with no distributional assumptions. This is optimal (matching the lower bound). The algorithm is remarkably simple: maintain weights, sample proportionally, and update multiplicatively based on losses.

✓ Checkpoint

Checkpoint 6: The MWU regret bound is $\eta T + \frac{\ln n}{\eta}$. For what value of η is this minimized, and what's the resulting regret?

Answer: Taking derivative: $T - \frac{\ln n}{\eta^2} = 0 \Rightarrow \eta = \sqrt{\frac{\ln n}{T}}$. Substituting: $\text{Regret} = \sqrt{\frac{\ln n}{T}} \cdot T + \frac{\ln n}{\sqrt{\frac{\ln n}{T}}} = 2\sqrt{T \ln n}$.

 Confidence Check

Self-Assessment: Check your understanding of MWU:

Rate your confidence (1-5):

- ☐ I can explain why MWU generalizes Randomized Weighted Majority
- ☐ I understand why losses in $[0, 1]$ are more general than binary mistakes
- ☐ I can derive the optimal learning rate $\eta = \sqrt{\frac{\ln n}{T}}$
- ☐ I see how the potential function proof extends to continuous losses
- ☐ I can explain intuitively why $O(\sqrt{T})$ regret is the “right” rate

If 3 or below: Review the MWU proof and compare it line-by-line to the Randomized Weighted Majority proof. The structure is identical!